

Section 1 :

Create Key :

The screenshot shows the AWS IAM console interface for creating an access key for the user 'YanisN'. The breadcrumb navigation at the top reads 'IAM > Users > YanisN > Create access key'. A green notification banner at the top states: 'This is the only time that the secret access key can be viewed or downloaded. You cannot recover it later. However, you can create a new access key any time.' On the left, a sidebar shows three steps: 'Step 1: Access key best practices & alternatives', 'Step 2 - optional: Set description tag', and 'Step 3: Retrieve access keys', with Step 3 being the active step. The main content area is titled 'Retrieve access keys' with an 'Info' link. It contains a section 'Access key' with a warning: 'If you lose or forget your secret access key, you cannot retrieve it. Instead, create a new access key and make the old key inactive.' Below this, there are two tabs: 'Access key' and 'Secret access key'. The 'Access key' tab shows the value 'AKIAVP3TQRNBFIS3JUDM'. The 'Secret access key' tab shows the value 'bTUaSB0exHPqxF2kASadQ2f3kZo91w9416yffG4' with a 'Hide' link. Below the tabs is a section 'Access key best practices' with a bulleted list: 'Never store your access key in plain text, in a code repository, or in code.', 'Disable or delete access key when no longer needed.', 'Enable least-privilege permissions.', and 'Rotate access keys regularly.' A link 'For more details about managing access keys, see the best practices for managing AWS access keys.' is provided. At the bottom right, there are two buttons: 'Download .csv file' and 'Done'.

Tester l'accès AWS sur Ubuntu :

```
yanis@Yessirr:~$ aws configure
AWS Access Key ID [None]: AKIAVP3TQRNBFIS3JUDM
AWS Secret Access Key [None]: bTUaSB0exHPqxF2kASadQ2f3kZo91w9416yffG4
Default region name [None]: eu-north-1
Default output format [None]: json
yanis@Yessirr:~$ aws sts get-caller-identity --region eu-north-1
{
  "UserId": "AIDAVP3TQRNBNS2GFPAOH",
  "Account": "377662507842",
  "Arn": "arn:aws:iam::377662507842:user/YanisN"
}
```

Section 2 :

Déployer une EC2 avec un script Bash :

```
yanis@Yessirr:~$ mkdir -p ~/devops_base/td2/scripts/bash
cd ~/devops_base/td2/scripts/bash
yanis@Yessirr:~/devops_base/td2/scripts/bash$ pwd
/home/yanis/devops_base/td2/scripts/bash
yanis@Yessirr:~/devops_base/td2/scripts/bash$ nano user-data.sh
yanis@Yessirr:~/devops_base/td2/scripts/bash$ nano deploy-ec2-instance.sh
yanis@Yessirr:~/devops_base/td2/scripts/bash$ nano deploy-ec2-instance.sh
yanis@Yessirr:~/devops_base/td2/scripts/bash$ chmod +x deploy-ec2-instance.sh
chmod +x user-data.sh
yanis@Yessirr:~/devops_base/td2/scripts/bash$ ./deploy-ec2-instance.sh
{
  "Return": true,
  "SecurityGroupRules": [
    {
      "SecurityGroupRuleId": "sgr-0e9e8f0ec21dd1bec",
      "GroupId": "sg-088d4fc5d53f18439c",
      "GroupOwnerId": "377662507842",
      "IsEgress": false,
      "IpProtocol": "tcp",
      "FromPort": 80,
      "ToPort": 80,
      "CidrIpv4": "0.0.0.0/0",
      "SecurityGroupRuleArn": "arn:aws:ec2:eu-north-1:377662507842:security-group-rule/sgr-0e9e8f0ec21dd1bec"
    }
  ]
}
Instance ID: i-0edae0814378bb8c5
Public IP: 13.60.239.201
Open http://13.60.239.201
```

Ouvre dans navigateur :

← → ↻ ⚠ Non sécurisé 13.60.239.201

Hello, World!

Nettoyage EC2 :

```
yanis@Yessirr:~$ aws ec2 describe-instances \
--region eu-north-1 \
--filters "Name=instance-state-name,Values=pending,running,stopping,stopped" \
--query "Reservations[].Instances[].InstanceId" \
--output table
-----
| DescribeInstances |
+-----+
| i-078dccba26867d9c1 |
+-----+
```

```

yanis@Yessirr:~$ aws ec2 terminate-instances \
--instance-ids i-078dccba26867d9c1 \
--region eu-north-1
{
  "TerminatingInstances": [
    {
      "InstanceId": "i-078dccba26867d9c1",
      "CurrentState": {
        "Code": 48,
        "Name": "terminated"
      },
      "PreviousState": {
        "Code": 80,
        "Name": "stopped"
      }
    }
  ]
}
yanis@Yessirr:~$ aws ec2 describe-instances \
--region eu-north-1 \
--filters "Name=instance-state-name,Values=pending,running,stopping,stopped" \
--query "Reservations[].Instances[].InstanceId" \
--output table

```

Section 3 :

Création du Playbook pour le deploiement de EC2 :

```

GNU nano 7.2 create-ec2.yml
- name: Deploy an EC2 instance in AWS
  hosts: localhost
  connection: local
  gather_facts: false

  environment:
    AWS_REGION: eu-north-1

  vars:
    region: eu-north-1
    instance_type: t3.micro
    sg_name: sample-app-ansible
    key_name: ansible-ch2
    tag_name: sample-app-ansible

  tasks:
    - name: Get latest Amazon Linux 2023 AMI
      amazon.aws.ec2_ami_info:
        owners: amazon
        filters:
          name: "al2023-ami-*-x86_64"
          region: "{{ region }}"
        register: ami_result

    - name: Create security group (HTTP 8080 + SSH 22)
      amazon.aws.ec2_security_group:
        name: "{{ sg_name }}"
        description: Allow HTTP and SSH traffic
        region: "{{ region }}"

```

Exécution du Playbook :

```

yaniso@vessir:~/devops/2024/scripts/ansible$ ansible-playbook create-ec2.yml
[WARNING]: No inventory was parsed, only implicit localhost is available
[WARNING]: provided hosts list is empty, only localhost is available. Note that the implicit localhost does not match 'all'

PLAY [Deploy an EC2 instance in AWS] *****

TASK [Get latest Amazon Linux 2023 AMI] *****
ok: [localhost]

TASK [Create security group (HTTP 8080 + SSH 22)] *****
changed: [localhost]

TASK [Create a new EC2 key pair] *****
changed: [localhost]

TASK [Launch EC2 instance] *****
changed: [localhost]

TASK [Show instance info] *****
ok: [localhost] => {
  "msg": [
    "InstanceId: i-04849fb5af7dc3f4",
    "Public IP: 51.21.224.133",
    "Try: http://51.21.224.133:8080"
  ]
}

PLAY RECAP *****
localhost : ok=5    changed=3    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0

```

Tester la connexion SSH :

```

yanis@Yessirr:~/devops_base/td2/scripts/ansible$ ls -l ansible-ch2.pem
-rw----- 1 yanis yanis 1678 Jan 18 15:15 ansible-ch2.pem
yanis@Yessirr:~/devops_base/td2/scripts/ansible$ ssh -i ansible-ch2.pem ec2-user@51.21.224.133
The authenticity of host '51.21.224.133 (51.21.224.133)' can't be established.
ED25519 key fingerprint is SHA256:XLVg0sdc/R+ER6ST7aMQsK0GRyzORaflJNWmEa63KtC.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '51.21.224.133' (ED25519) to the list of known hosts.

#_
~\_#####_
~~\_#####\
~~\#####|
~~\##/\
~~\#/--- Amazon Linux 2023 (ECS Optimized)
~~V~' ->
~~~
~~~.
~~~\_/
~~~/_/m/'

```

Création de l'inventaire Ansible :

```
yanis@Yessirr:~/devops_base/td2/scripts/ansible$ nano inventory.aws_ec2.yml
yanis@Yessirr:~/devops_base/td2/scripts/ansible$ ansible-inventory -i inventory.aws_ec2.yml --graph
@all:
|--@ungrouped:
|--@aws_ec2:
|   |--ec2-51-21-224-133.eu-north-1.compute.amazonaws.com
|   |--ec2-51-20-140-166.eu-north-1.compute.amazonaws.com
|--@_ch2_instances:
|   |--ec2-51-21-224-133.eu-north-1.compute.amazonaws.com
```

```

GNU nano 7.2
plugin: amazon.aws.aws_ec2
regions:
  - eu-north-1

keyed_groups:
  - key: tags.Ansible
    leading_separator: ""

```

Création du groupe de variables Ansible :

```

yanis@Yessirr:~/devops_base/td2/scripts/ansible$ mkdir -p group_vars
nano group_vars/ch2_instances.yml

```

Création du Playbook de configuration Ansible :

```

GNU nano 7.2                                configure_sample_app_playbook.yml
- name: Configure the EC2 instance to run a sample app
  hosts: ch2_instances
  gather_facts: true
  become: true
  roles:
    - sample-app

```

Création du rôle Ansible :

```

GNU nano 7.2                                roles/sample-app/tasks/main.yml
- name: Add Node packages to yum
  shell: |
    curl -fsSL https://rpm.nodesource.com/setup_18.x | bash -
    dnf install -y nodejs
  args:
    executable: /bin/bash

- name: Create app directory
  file:
    path: /opt/sample-app
    state: directory

- name: Copy app.js
  copy:
    src: app.js
    dest: /opt/sample-app/app.js
    mode: "0644"

- name: Start app on port 8080
  shell: |
    nohup node /opt/sample-app/app.js > /var/log/sample-app.log 2>&1 &
  args:
    executable: /bin/bash

```

Exécution du Playbook :

```
yanis@Yessirr:~/devops_base/td2/scripts/ansible$ ansible-playbook -i inventory.aws_ec2.yml configure_sample_app_playbook.yml

PLAY [Configure the EC2 instance to run a sample app] *****

TASK [Gathering Facts] *****
[WARNING]: Platform linux on host ec2-51-21-224-133.eu-north-1.compute.amazonaws.com is using the discovered Python
interpreter at /usr/bin/python3.9, but future installation of another Python interpreter could change the meaning of that
path. See https://docs.ansible.com/ansible-core/2.16/reference_appendices/interpreter_discovery.html for more information.
ok: [ec2-51-21-224-133.eu-north-1.compute.amazonaws.com]

TASK [sample-app : Add Node packages to yum] *****
changed: [ec2-51-21-224-133.eu-north-1.compute.amazonaws.com]

TASK [sample-app : Create app directory] *****
changed: [ec2-51-21-224-133.eu-north-1.compute.amazonaws.com]

TASK [sample-app : Copy app.js] *****
changed: [ec2-51-21-224-133.eu-north-1.compute.amazonaws.com]

TASK [sample-app : Start app on port 8080] *****
changed: [ec2-51-21-224-133.eu-north-1.compute.amazonaws.com]

PLAY RECAP *****
ec2-51-21-224-133.eu-north-1.compute.amazonaws.com : ok=5  changed=4  unreachable=0  failed=0  skipped=0  rescued
=0  ignored=0
```

Ouvrir sur navigateur :

<http://51.21.224.133:8080>



Hello, World!

Exercice 3 :

Les tâches Ansible sont idempotentes : l'exécution multiple du même playbook ne réapplique pas les modifications si le système est déjà dans l'état souhaité. Lors de la deuxième exécution, Ansible détecte que Node.js, les fichiers d'application et le service en cours d'exécution sont déjà présents, donc aucune modification n'est nécessaire.

Exercice 4 :

Pour déployer plusieurs instances EC2, le playbook de création EC2 peut être modifié afin de lancer plusieurs instances à l'aide du paramètre count. L'inventaire AWS dynamique détectera automatiquement toutes les instances dotées des balises appropriées et les regroupera sous ch2_instances. Le playbook de configuration peut ensuite être appliqué simultanément à toutes les instances sans modification.

Nettoyage AWS :

```
yanis@Yessirr:~/devops_base/td2/scripts/ansible$ aws ec2 terminate-instances --region eu-north-1 --instance-ids i-04849fb5af7dcb3f4
aws ec2 delete-key-pair --region eu-north-1 --key-name ansible-ch2
aws ec2 delete-security-group --region eu-north-1 --group-name sample-app-ansible
rm -f ansible-ch2.pem
{
  "TerminatingInstances": [
    {
      "InstanceId": "i-04849fb5af7dcb3f4",
      "CurrentState": {
        "Code": 48,
        "Name": "terminated"
      },
      "PreviousState": {
        "Code": 48,
        "Name": "terminated"
      }
    }
  ]
}
{
  "Return": true,
  "KeyPairId": "key-03d4aefb3df9ea7d8"
}
{
  "Return": true,
  "GroupId": "sg-0e54ed58c0f9881d7"
}
```

Avant de passer à la prochaine section, on a remarqué qu'il y avait une instance EC2 encore en exécution :

```
yanis@Yessirr:~/devops_base/td2/scripts/ansible$ aws ec2 describe-instances \
--region eu-north-1 \
--filters "Name=instance-state-name,Values=running" \
--query "Reservations[].Instances[].{Id:InstanceId,IP:PublicIpAddress,Name:Tags[?Key=='Name']|[0].Value}" \
--output table
```

IP	Id	Name
51.20.140.166	i-0dc9975d6c1ae6f32	ansible-sample-app

On a juste terminer l'instance :

```
yanis@Yessirr:~/devops_base/td2/scripts/ansible$ aws ec2 terminate-instances \
--region eu-north-1 \
--instance-ids i-0dc9975d6c1ae6f32
{
  "TerminatingInstances": [
    {
      "InstanceId": "i-0dc9975d6c1ae6f32",
      "CurrentState": {
        "Code": 32,
        "Name": "shutting-down"
      },
      "PreviousState": {
        "Code": 16,
        "Name": "running"
      }
    }
  ]
}
yanis@Yessirr:~/devops_base/td2/scripts/ansible$ aws ec2 describe-instances \
--region eu-north-1 \
--filters "Name=instance-state-name,Values=running" \
--output table
```

DescribeInstances		
-------------------	--	--

Section 4 :

On a créé le dossier Packer et on a copié app.js dedans :

```
yanis@Yessirr:~/devops_base/td2/scripts/ansible$ mkdir -p ~/devops_base/td2/scripts/packer
cd ~/devops_base/td2/scripts/packer
yanis@Yessirr:~/devops_base/td2/scripts/packer$ cp ~/devops_base/td2/scripts/ansible/roles/sample-app/files/app.js ~/devops_base/td2/scripts/packer/
```

On a créé le Template Packer en le modifiant des parties pour qu'il puisse bien fonctionner. On a modifié le source ami :

```
source_ami_filter {
  filters = {
    name                = "al2023-ami-*-x86_64"
    root-device-type    = "ebs"
    virtualization-type = "hvm"
  }
  owners      = ["amazon"]
  most_recent = true
}

ssh_username = "ec2-user"
}
```

On a modifié le provisioner "shell" également :

```
provisioner "shell" {
  inline = [
    "set -euxo pipefail",
    "sudo dnf -y update",

    # On n'installe PAS curl (sinon conflit curl-minimal)
    "sudo dnf -y install tar xz",

    "cd /tmp",
    "curl -fsSL0 https://nodejs.org/dist/v20.11.1/node-v20.11.1-linux-x64.tar.xz",
    "sudo mkdir -p /usr/local/lib/nodejs",
    "sudo tar -xJf node-v20.11.1-linux-x64.tar.xz -C /usr/local/lib/nodejs",

    "sudo ln -sf /usr/local/lib/nodejs/node-v20.11.1-linux-x64/bin/node /usr/local/bin/node",
    "sudo ln -sf /usr/local/lib/nodejs/node-v20.11.1-linux-x64/bin/npm /usr/local/bin/npm",
    "sudo ln -sf /usr/local/lib/nodejs/node-v20.11.1-linux-x64/bin/npx /usr/local/bin/npx",

    "node -v",
    "npm -v"
  ]
  pause_before = "10s"
}
```

On a ensuite initialisé le Packer et enfin builder l'AMI :

```
yanis@Yessirr:~/devops_base/td2/scripts/packer$ packer init sample-app.pkr.hcl
Installed plugin github.com/hashicorp/amazon v1.8.0 in "/home/yanis/.config/packer/plugins/github.com/hashicorp/amazon/packer-plugin-amazon_v1.8.0_x5.0_linux_amd64"
```



```
yanis@Yessirr:~/devops_base/td2/scripts/packer$ packer fmt sample-app.pkr.hcl
packer validate sample-app.pkr.hcl
packer build sample-app.pkr.hcl
sample-app.pkr.hcl
The configuration is valid.
```

Quand le build a réussi, on a l'AMI qui est créé : *ami-0b05904fe9219b67d*

```
Build 'amazon-ebs.amazon_linux' finished after 6 minutes 49 seconds.

==> Wait completed after 6 minutes 49 seconds

==> Builds finished. The artifacts of successful builds are:
--> amazon-ebs.amazon_linux: AMIs were created:
eu-north-1: ami-0b05904fe9219b67d
```

On en aura besoin dans la prochaine section.

Exercice 5 :

Relancer la commande `packer build` va recréer une nouvelle AMI à chaque exécution. Le nom de l'image étant unique grâce à `uuidv4()`, Packer recrée une instance temporaire, applique les configurations, puis génère une nouvelle image avec un identifiant différent.

Exercice 6 :

Les builders définissent la plateforme cible et les paramètres de création de l'image. Dans ce projet, le builder `amazon-ebs` permet de créer une AMI sur AWS. Les provisioners servent à configurer l'instance pendant le build, notamment à copier l'application et à installer les dépendances nécessaires.

Section 5 :

Après avoir créé le dossier `ec2-instance`, on y ajoute les fichiers `main.tf`, `variables.tf`, `user-data.tf`, `outputs.tf`.

Ensuite on lance le tofu avec le AMI ID qu'on a eu à la section précédente :

```

yanis@Yessirr:~/devops_base/td2/scripts/tofu/ec2-instance$ tofu apply -var ami_id=ami-0b05904fe9219b67d
aws_security_group.sample_app: Refreshing state... [id=sg-0dfe5a51c8a931174]
aws_security_group_rule.allow_http_inbound: Refreshing state... [id=sgrule-4017220979]

OpenTofu used the selected providers to generate the following execution plan. Resource actions are indicated with the following
symbols:
+ create

OpenTofu will perform the following actions:

# aws_instance.sample_app will be created
+ resource "aws_instance" "sample_app" {
  + ami              = "ami-0b05904fe9219b67d"
  + arn              = (known after apply)
  + associate_public_ip_address = (known after apply)
  + availability_zone = (known after apply)
  + disable_api_stop  = (known after apply)
  + disable_api_termination = (known after apply)
  + ebs_optimized     = (known after apply)
  + enable_primary_ipv6 = (known after apply)

```

On trouve à la fin l'IP Publique pour pouvoir ouvrir le navigateur.

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.

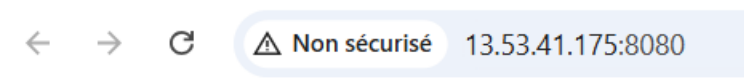
Outputs:

```

instance_id = "i-0e0df0e7d9d75c42c"
public_ip   = "13.53.41.175"
security_group_id = "sg-0dfe5a51c8a931174"

```

<http://13.53.41.175:8080>



Hello, World!

On a modifié le main.tf et run le tofu une nouvelle fois :

```

yanis@Yessirr:~/devops_base/td2/scripts/tofu/ec2-instance$ tofu apply -var ami_id=ami-0b05904fe9219b67d
aws_security_group.sample_app: Refreshing state... [id=sg-0dfe5a51c8a931174]
aws_security_group_rule.allow_http_inbound: Refreshing state... [id=sgrule-4017220979]
aws_instance.sample_app: Refreshing state... [id=i-0e0df0e7d9d75c42c]

OpenTofu used the selected providers to generate the following execution plan. Resource actions are indicated with the following
symbols:
~ update in-place (current -> planned)

```

Enfin on a détruit les ressources :

```

yanis@Yessirr:~/devops_base/td2/scripts/tofu/ec2-instance$ tofu destroy -var ami_id=ami-0b05904fe9219b67d
aws_security_group.sample_app: Refreshing state... [id=sg-0dfe5a51c8a931174]
aws_security_group_rule.allow_http_inbound: Refreshing state... [id=sgrule-4017220979]
aws_instance.sample_app: Refreshing state... [id=i-0e0df0e7d9d75c42c]

OpenTofu used the selected providers to generate the following execution plan. Resource actions are indicated with the following
symbols:
- destroy

OpenTofu will perform the following actions:

# aws_instance.sample_app will be destroyed

```

```

aws_security_group_rule.allow_http_inbound: Destroying... [id=sgrule-4017220979]
aws_instance.sample_app: Destroying... [id=i-0e0df0e7d9d75c42c]
aws_security_group_rule.allow_http_inbound: Destruction complete after 0s
aws_instance.sample_app: Still destroying... [id=i-0e0df0e7d9d75c42c, 10s elapsed]
aws_instance.sample_app: Still destroying... [id=i-0e0df0e7d9d75c42c, 20s elapsed]
aws_instance.sample_app: Still destroying... [id=i-0e0df0e7d9d75c42c, 30s elapsed]
aws_instance.sample_app: Still destroying... [id=i-0e0df0e7d9d75c42c, 41s elapsed]
aws_instance.sample_app: Still destroying... [id=i-0e0df0e7d9d75c42c, 51s elapsed]
aws_instance.sample_app: Still destroying... [id=i-0e0df0e7d9d75c42c, 1m1s elapsed]
aws_instance.sample_app: Destruction complete after 1m1s
aws_security_group.sample_app: Destroying... [id=sg-0dfe5a51c8a931174]
aws_security_group.sample_app: Destruction complete after 1s

Destroy complete! Resources: 3 destroyed.

```

Exercice 7 :

Après l'exécution de `tofu destroy`, les ressources sont supprimées. Si on relance `tofu apply`, OpenTofu recrée automatiquement toutes les ressources définies dans la configuration, car elles n'existent plus dans l'infrastructure.

Exercice 8 :

Pour déployer plusieurs instances, on peut utiliser une boucle avec `count` (ou `for_each`) afin d'éviter de dupliquer le code. Il suffit d'ajouter une variable pour le nombre d'instances, puis d'utiliser `count` sur la ressource `aws_instance`.

Section 6 :

On crée le dossier `modules` et déplace le module dedans. On a créé aussi `live/sample-app` dans lequel on a créé un `main.tf`

```

yanis@Yessirr:~/devops_base/td2/scripts/tofu/ec2-instance$ mkdir -p ~/devops_base/td2/scripts/tofu/modules
yanis@Yessirr:~/devops_base/td2/scripts/tofu/ec2-instance$ mv ~/devops_base/td2/scripts/tofu/ec2-instance \
~/devops_base/td2/scripts/tofu/modules/ec2-instance
yanis@Yessirr:~/devops_base/td2/scripts/tofu/ec2-instance$ mkdir -p ~/devops_base/td2/scripts/tofu/live/sample-app
yanis@Yessirr:~/devops_base/td2/scripts/tofu/ec2-instance$ cd ~/devops_base/td2/scripts/tofu/live/sample-app
yanis@Yessirr:~/devops_base/td2/scripts/tofu/live/sample-app$ nano main.tf

```

Dans le dossier `modules`, on a modifié dans le `main.tf`, tous les noms `"sample-app-tofu"` par `var.name` :

```

provider "aws" {
  region = "eu-north-1"
}

resource "aws_security_group" "sample_app" {
  name       = "var.name"
  description = "Allow HTTP traffic into ${var.name}"
}

```

```

tags = {
  Name = "var.name"
  Test = "update"
}

```

Les dossiers sont organisés comme ça pour sample-app et ec2-modules:

... Ubuntu > home > yanis > devops_base > td2 > scripts > tofu > live > sample-app >

Nom	Modifié le	Type	Taille
📁 .terraform	19/01/2026 21:18	Dossier de fichiers	
📄 .terraform.lock.hcl	19/01/2026 21:03	Fichier HCL	1 Ko
📄 main.tf	19/01/2026 21:13	Fichier TF	1 Ko
📄 outputs.tf	19/01/2026 21:49	Fichier TF	1 Ko
📄 terraform.tfstate	19/01/2026 21:49	Fichier TFSTATE	10 Ko
📄 terraform.tfstate.backup	19/01/2026 21:49	Fichier BACKUP	10 Ko
📄 variables.tf	19/01/2026 21:12	Fichier TF	1 Ko

> ... Ubuntu > home > yanis > devops_base > td2 > scripts > tofu > modules > ec2-instance >

Nom	Modifié le	Type	Taille
📁 .terraform	19/01/2026 19:04	Dossier de fichiers	
📄 .terraform.lock.hcl	19/01/2026 19:04	Fichier HCL	1 Ko
📄 main.tf	19/01/2026 21:17	Fichier TF	1 Ko
📄 outputs.tf	19/01/2026 19:01	Fichier TF	1 Ko
📄 terraform.tfstate	19/01/2026 19:51	Fichier TFSTATE	1 Ko
📄 terraform.tfstate.backup	19/01/2026 19:51	Fichier BACKUP	5 Ko
📄 user-data.sh	19/01/2026 19:00	sh_auto_file	1 Ko
📄 variables.tf	19/01/2026 20:36	Fichier TF	1 Ko

Quand on exécute le tofu avec le AMI ID, on a en sortie les IPs qu'on peut ouvrir sur le navigateur :

```

yanis@Yessirr:~/devops_base/td2/scripts/tofu/live/sample-app$ tofu apply -var ami_id=ami-0b05904fe9219b67d
module.sample_app_1.aws_security_group.sample_app: Refreshing state... [id=sg-0b9eaddaf83c7986c]
module.sample_app_2.aws_security_group.sample_app: Refreshing state... [id=sg-09fcfb4da2601a76a]
module.sample_app_2.aws_security_group_rule.allow_http_inbound: Refreshing state... [id=sgrule-419411105]
module.sample_app_1.aws_security_group_rule.allow_http_inbound: Refreshing state... [id=sgrule-3827495856]
module.sample_app_2.aws_instance.sample_app: Refreshing state... [id=i-0bb23e62411e14355]
module.sample_app_1.aws_instance.sample_app: Refreshing state... [id=i-0e07d59a285570e74]

```

Outputs:

```

app_url_1 = "http://16.170.206.146:8080"
app_url_2 = "http://16.171.11.94:8080"

```

← → ↻ ⚠ Non sécurisé 16.170.206.146:8080

Hello, World!

Exercice 9 :

On doit modifier le module pour le rendre plus flexible en paramétrant le type d'instance et le port.

On ajoute les variables d'instance et le port dans variables.tf :

```

variable "ami_id" {
  description = "The ID of the AMI to run."
  type       = string
}

variable "name" {
  description = "The base name for the instance and all other resources"
  type       = string
}

variable "instance_type" {
  description = "EC2 instance type"
  type       = string
  default    = "t3.micro"
}

variable "port" {
  description = "Port to open for the app"
  type       = number
  default    = 8080
}

```

Dans le main, on a remplacé 8080 par var.port et dans l'instance on remplace le type par var.instance_type :

```
resource "aws_security_group_rule" "allow_http_inbound" {
  type           = "ingress"
  protocol       = "tcp"
  from_port      = var.port
  to_port        = var.port
  security_group_id = aws_security_group.sample_app.id
  cidr_blocks    = ["0.0.0.0/0"]
}
```

```
resource "aws_instance" "sample_app" {
  ami            = var.ami_id
  instance_type  = var.instance_type
  vpc_security_group_ids = [aws_security_group.sample_app.id]
  user_data      = file("${path.module}/user-data.sh")
}
```

Dans le main de sample-app, dans chaque module, on a ajouté :

```
instance_type = "t3.micro"
```

```
port          = 8080
```

```
module "sample_app_1" {
  source = "/home/yanis/devops_base/td2/scripts/tofu/modules/ec2-instance"
  ami_id = var.ami_id
  name    = "sample-app-tofu-1"
  instance_type = "t3.micro"
  port      = 8080
}

module "sample_app_2" {
  source = "/home/yanis/devops_base/td2/scripts/tofu/modules/ec2-instance"
  ami_id = var.ami_id
  name    = "sample-app-tofu-2"
  instance_type = "t3.micro"
  port      = 8080
}
```

```
Apply complete! Resources: 6 added, 0 changed, 0 destroyed.
```

Outputs:

```
app_url_1 = "http://16.171.236.239:8080"
app_url_2 = "http://13.48.138.108:8080"
```

Exercice 10 :

Maintenant, au lieu d'avoir module "sample_app_1" et module "sample_app_2", on a fait un seul bloc module

On met outputs à jour :

Dans outputs.tf on utilisait module.sample_app_1 / module.sample_app_2, maintenant on le remplace par un output en map.

```
output "app_url_1" {
  description = "URL of sample-app instance 1"
  value       = "http://${module.sample_app_1.public_ip}:8080"
}

output "app_url_2" {
  description = "URL of sample-app instance 2"
  value       = "http://${module.sample_app_2.public_ip}:8080"
}

output "public_ips" {
  value = { for k, m in module.sample_app : k => m.public_ip }
}
```

```
Apply complete! Resources: 6 added, 0 changed, 0 destroyed.
```

Outputs:

```
app_urls = {
  "sample-app-1" = "http://13.51.235.97:8080"
  "sample-app-2" = "http://13.60.90.229:8080"
}
public_ips = {
  "sample-app-1" = "13.51.235.97"
  "sample-app-2" = "13.60.90.229"
}
```

Section 6 :

Dans cette section, le module OpenTofu a été externalisé dans un dépôt GitHub public. Le module est ensuite référencé depuis le module racine à l'aide de l'attribut source. Lors de l'exécution de `tofu init`, OpenTofu télécharge automatiquement le module distant. Le déploiement est ensuite réalisé avec `tofu apply`, puis les ressources sont détruites avec `tofu destroy` afin d'éviter tout coût inutile.

Dans le main, on a modifié la source :

```
module "sample_app" {  
  for_each      = var.instances  
  source        = "github.com/yanis-nouili/devops_base.git//scripts/tofu/modules/ec2-instance"  
  ami_id        = var.ami_id  
  name          = each.key  
  instance_type = each.value.instance_type  
  port          = each.value.port  
}
```

```
yanis@Yessirr:~/devops_base/td2/scripts/tofu/live/sample-app$ rm -rf .terraform .terraform.lock.hcl  
tofu init  
  
Initializing the backend...  
Initializing modules...  
Downloading git::https://github.com/yanis-nouili/devops_base.git for sample_app...  
- sample_app in .terraform/modules/sample_app/scripts/tofu/modules/ec2-instance  
  
Initializing provider plugins...  
- Finding latest version of hashicorp/aws...  
- Installing hashicorp/aws v6.28.0...  
- Installed hashicorp/aws v6.28.0 (signed, key ID 0C0AF313E5FD9F80)  
  
Providers are signed by their developers.  
If you'd like to know more about provider signing, you can read about it here:  
https://opentofu.org/docs/cli/plugins/signing/  
  
OpenTofu has created a lock file .terraform.lock.hcl to record the provider  
selections it made above. Include this file in your version control repository  
so that OpenTofu can guarantee to make the same selections by default when  
you run "tofu init" in the future.  
  
OpenTofu has been successfully initialized!
```



```

yanis@Yessirr:~/devops_base/td2/scripts/tofu/live/sample-app$ tofu apply -var ami_id=ami-0b05904fe9219b67d
module.sample_app["sample-app-2"].aws_security_group.sample_app: Refreshing state... [id=sg-08009d771a9d6f186]
module.sample_app["sample-app-1"].aws_security_group.sample_app: Refreshing state... [id=sg-055860a56b05dcbac]
module.sample_app["sample-app-1"].aws_security_group_rule.allow_http_inbound: Refreshing state... [id=sgrule-1528864872]
module.sample_app["sample-app-2"].aws_security_group_rule.allow_http_inbound: Refreshing state... [id=sgrule-2619848492]
module.sample_app["sample-app-1"].aws_instance.sample_app: Refreshing state... [id=i-0b488a655d54d1603]
module.sample_app["sample-app-2"].aws_instance.sample_app: Refreshing state... [id=i-08eb341548de5fadd]

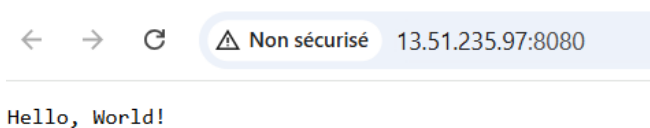
No changes. Your infrastructure matches the configuration.

OpenTofu has compared your real infrastructure against your configuration and found no differences, so no changes are needed.

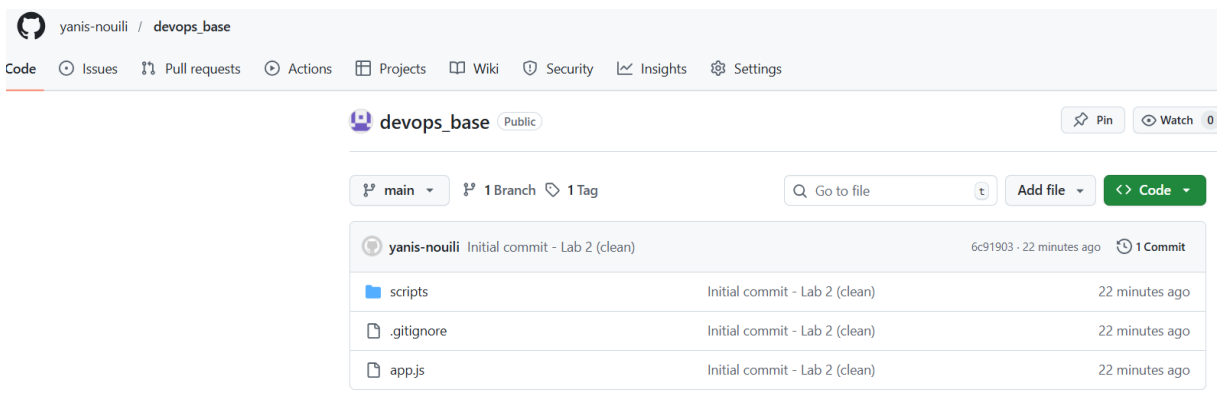
Apply complete! Resources: 0 added, 0 changed, 0 destroyed.

Outputs:
app_urls = {
  "sample-app-1" = "http://13.51.235.97:8080"
  "sample-app-2" = "http://13.60.90.229:8080"
}
public_ips = {
  "sample-app-1" = "13.51.235.97"
  "sample-app-2" = "13.60.90.229"
}

```



Environnement Github :



Exercice 11 :

On a créé un tag sur GitHub dans notre repo local. On a ensuite utilisé ce tag dans source :

```

module "sample_app" {
  for_each = var.instances
  source = "github.com/yanis-nouili/devops_base.git//scripts/tofu/modules/ec2-instance?ref=v1.0.0"
  ami_id   = var.ami_id
  name     = each.key
}

```

On a re-init et apply :

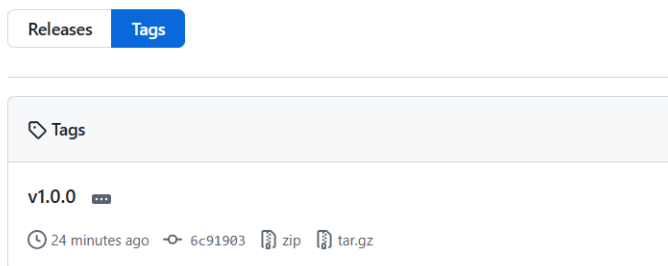
```
yanis@Yessir:~/devops_base/td2/scripts/tofu/live/sample-app$ cd ~/devops_base/td2/scripts/tofu/live/sample-app
rm -rf .terraform .terraform.lock.hcl
tofu init
tofu apply -var ami_id=ami-0b05904fe9219b67d

Initializing the backend...
Initializing modules...
Downloading git::https://github.com/yanis-nouili/devops_base.git?ref=v1.0.0 for sample_app...
- sample_app in .terraform/modules/sample_app/scripts/tofu/modules/ec2-instance

Initializing provider plugins...
- Finding latest version of hashicorp/aws...
- Installing hashicorp/aws v6.28.0...
- Installed hashicorp/aws v6.28.0 (signed, key ID 0C0AF313E5FD9F80)

Providers are signed by their developers.
If you'd like to know more about provider signing, you can read about it here:
https://opentofu.org/docs/cli/plugins/signing/

OpenTofu has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that OpenTofu can guarantee to make the same selections by default when
you run "tofu init" in the future.
```



Le module étant hébergé sur GitHub, un mécanisme de versionning a été mis en place via un tag (v1.0.1), nous l'avons modifié entre temps. Le root module référence alors une version stable du module grâce au paramètre ?ref=v1.0.1. Cela garantit la reproductibilité et évite qu'un changement futur du module casse le déploiement.